

MÓDULO 02: Estructuras de Datos en Python (Listas, Tuplas, Diccionarios y Conjuntos)

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora 30 minutos

Módulo: 02 - Data Structures: Lists, Tuples, Dictionaries & Sets

1. Objetivos de Aprendizaje

1. **Listas (`list`):** Dominar colecciones ordenadas y mutables, métodos de manipulación (`append` , `extend` , `pop` , `sort`) y técnicas avanzadas de rebanado (`slicing` :) unidimensional y multidimensional.
2. **Tuplas (`tuple`):** Comprender la inmutabilidad estricta, desempaqueado de variables, conversiones explícitas y tuplas compuestas.
3. **Diccionarios (`dict`):** Estructurar datos complejos en pares clave-valor, iterar mediante `.items()` , `.keys()` , `.values()` y gestionar estructuras anidadas.
4. **Conjuntos (`set`):** Aplicar álgebra de conjuntos (unión `|` , intersección `&` , diferencia `-`) para filtrado de datos únicos y alineación de atributos científicos.
5. **Caso Práctico Integrador:** Análisis de datos ornitológicos y taxonomía biológica combinando diccionarios anidados y operaciones de conjuntos.

 Cabecera del cuaderno

SESION 2: Estructuras de Datos en Python

Este cuaderno está diseñado para ayudarte a entender los tipos de estructuras de datos nativas en Python y cómo trabajar con ellas usando Jupyter Notebooks en Visual Studio Code.

@Gustavo Liñán Cembrano

Listas en Python

Las listas son estructuras de datos que permiten almacenar múltiples elementos en una sola variable. Son mutables, lo que significa que sus elementos pueden cambiarse después de su creación.

- **PARA NO OLVIDAR JAMÁS: Los índices de los elementos de una lista SIEMPRE se numeran desde 0:**

```
MiLista = [5,6,7]

print(MiLista[0])# Resultado 5
print(MiLista[1])# Resultado 6
print(MiLista[2])# Resultado 7
print(MiLista[3])# Te va a dar este error!!!!

-----
IndexError                                Traceback (most recent
call last)
```

Características principales

Se definen con corchetes []. Pueden contener elementos de cualquier tipo: enteros, cadenas, booleanos, otras listas, etc. Se pueden modificar, agregar, eliminar y ordenar elementos.

Operaciones comunes con listas

Operación	Código	Descripción
Crear lista	<code>mi_lista = [1, 2, 3]</code>	Lista con tres enteros
Acceder	<code>mi_lista[0]</code>	Primer elemento
Modificar	<code>mi_lista[1] = 99</code>	Cambia el segundo elemento
Agregar	<code>mi_lista.append(4)</code>	Añade al final
Encontrar	<code>mi_lista.index(2)</code>	Devuelve la posición del elemento 2
Insertar	<code>mi_lista.insert(1, 'a')</code>	Inserta 'a' en la posición 1
Eliminar	<code>mi_lista.remove(99)</code>	Elimina el valor 99
Pop	<code>mi_lista.pop()</code>	Elimina y devuelve el último elemento
Longitud	<code>len(mi_lista)</code>	Número de elementos en la lista
Ordenar	<code>mi_lista.sort()</code>	Ordena la lista en orden ascendente
Reversar	<code>mi_lista.reverse()</code>	Invierte el orden de los elementos

Operación	Código	Descripción
Copiar	<code>nueva_lista = mi_lista.copy()</code>	Crea una copia superficial de la lista
Extender	<code>mi_lista.extend([5, 6])</code>	Añade múltiples elementos al final
Contar	<code>mi_lista.count(2)</code>	Cuenta cuántas veces aparece el valor 2
Borrar por índice	<code>del mi_lista[0]</code>	Elimina el elemento en la posición 0
Vaciar lista	<code>mi_lista.clear()</code>	Elimina todos los elementos de la lista

```
In [ ]: # Operaciones basicas con listas

# Crear una lista
frutas = ["manzana", "banana", "cereza"]
print(frutas)
```

```
In [ ]: # Acceder a elementos
print(frutas[0]) #Recuerda!!! Los indices empiezan por 0
```

```
In [ ]: # Modificar un elemento
frutas[1] = "naranja"
print(frutas)
```

```
In [ ]: # Agregar un elemento
frutas.append("kiwi")
print(frutas)
```

```
In [ ]: # Insertar en una posición específica
frutas.insert(1, "pera")
print(frutas)
```

```
In [ ]: # Eliminar por valor
frutas.remove("cereza")
print(frutas)
```

```
In [ ]: # Eliminar por índice
eliminado = frutas.pop(1)
print(f'Eliminado: {eliminado}')
print(frutas)
```

```
In [ ]: # Buscar índice de un elemento
indice = frutas.index("kiwi")
print(f'Índice de kiwi: {indice}')
```

```
In [ ]: # Ordenando listas ordenables
numeros = [5, 3, 8, 1, 9]

# Ordenar
```

```
numeros.sort()
print(f'Ordenados: {numeros}')

# Invertir
numeros.reverse()
print(f'Invertidos: {numeros}')
```

```
In [ ]: # Listas anidadas y acceso por niveles
        # En python las listas anidadas se definen fila a fila!!!

matriz = [[1, 2], [3, 4], [5, 6]]
print(matriz)
```

```
In [ ]: # Acceder a elementos internos primero fila, despues columna,
        # RECUERDA LOS INDICES DE DIRECCION SIEMPRE EMPIEZAN POR 0

print(matriz[1][0]) # ?????
```

```
In [ ]: # CONTENIDO AVANZADO ...YA LO VEREMOS!!!
        # Iterar sobre elementos internos
        #
        for LoQueQUIERO in matriz:
            print(LoQueQUIERO)
```

```
In [ ]: for fila in matriz:
        for valor in fila:
            print(valor)
        print("----") #Cada vez que imprima una fila, pongo los ---
```

Uso de `:` en Python (slicing)

El operador `:` se utiliza para **obtener sublistas, subtuplas o subcadenas** a partir de una secuencia en Python.

Esto se conoce como **slicing** o corte de secuencias. La sintaxis UNIVERSAL es

`lista[inicio:fin:paso]`

inicio → índice donde empieza la sublista (inclusive).

fin → índice donde termina la sublista (exclusive).

paso → cada cuántos elementos se toma uno (puede ser negativo).

Si dejamos inicio o fin vacíos, Python asume el inicio o fin de la lista.

```
In [ ]: numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

        #imprimirlos todos

print(numeros[:]) #Es exactamente lo mismo que print(numeros)
```

```

In [ ]: # Imprimir la lista al revés

print(numeros[::-1])

# Explicación:
# - Inicio vacío: empieza desde el primer elemento de la lista.
# - Final vacío: termina en el último elemento de la lista.
# - Paso -1: recorre la lista de derecha a izquierda, creando una nueva lista
#   No es que "reste 1 al índice 0", sino que Python recorre los elementos e

In [ ]: # Obtener una sublista del índice 2 al 5 (5 no incluido)
print(numeros[2:5]) # salida: [2, 3, 4]

In [ ]: # Obtener una sublista del índice 2 al 8 (saltando 3)
print(numeros[2:8:3])

In [ ]: # Desde el inicio hasta el índice 4
print(numeros[:5]) # salida: [0, 1, 2, 3, 4]

In [ ]: # Desde el índice 5 hasta el final
print(numeros[5:]) # salida: [5, 6, 7, 8, 9]

In [ ]: # Sería lo mismo que
print(numeros[5::])
#ya que por defecto el step es 1 y no hay que ponerlo

In [ ]: # Cada dos elementos
print(numeros[::2]) # salida: [0, 2, 4, 6, 8]

In [ ]: # Sublista invertida de dos en dos
print(numeros[::-2]) # salida: [9, 7, 5, 3, 1]

In [ ]: texto = "Python"
print(texto[1:4]) # 'yth' (del índice 1 al 3)
print(texto[:3]) # 'Pyt'
print(texto[3:]) # 'hon'
print(texto[::-1]) # 'nohtyP'

```

Ejercicios guiados: Listas en Python

Nivel 1: Operaciones básicas

Ejercicio 1.1

Crea una lista llamada `colores` que contenga tres colores (rojo, verde y azul). Luego imprime el segundo color.

Ejercicio 1.2

Modifica el primer color de la lista por `"verde"` y vuelve a imprimir la lista.

```
In [ ]: #tu codigo aqui
```

Nivel 2: Inserción, eliminación y búsqueda

Ejercicio 2.1

Agrega el color "morado" al final de la lista `colores`. Luego inserta "amarillo" en la segunda posición.

Ejercicio 2.2

Elimina el color "verde" de la lista. Luego elimina el último elemento usando `pop()` y guarda el valor eliminado en una variable.

Ejercicio 2.3

Crea una lista de números y encuentra el índice del número 42.

```
In [ ]:
```

Nivel 3: Ordenación y reverso

Ejercicio 3.1

Crea una lista de números desordenados. Ordénala de menor a mayor y luego inviértela.

```
In [ ]: #tu codigo aqui
```

Tuplas en Python

Las tuplas son estructuras de datos similares a las listas, pero con una diferencia clave:

Esto significa que, una vez creada una tupla, no se puede modificar (no se pueden añadir, eliminar ni cambiar elementos).

Características principales

- Se definen con paréntesis `()`, aunque los paréntesis pueden omitirse en muchos casos.
- Pueden contener elementos de **cualquier tipo**.
- Son útiles cuando se necesita una colección de datos que no debe cambiar.
- Se pueden **desempaquetar** fácilmente en variables.

Operaciones comunes

Operación	Código	Descripción
Crear tupla	<code>mi_tupla = (1, 2, 3)</code>	Tupla con tres enteros
Acceder	<code>mi_tupla[0]</code>	Primer elemento
Longitud	<code>len(mi_tupla)</code>	Número de elementos
Desempaquetar	<code>a, b, c = mi_tupla</code>	Asigna cada elemento a una variable
Concatenar	<code>mi_tupla + (4, 5)</code>	Crea una nueva tupla
Repetir	<code>mi_tupla * 2</code>	Repite los elementos
Verificar existencia	<code>2 in mi_tupla</code>	Devuelve <code>True</code> si el elemento está
Conversión	<code>tuple([1, 2, 3])</code>	Convierte una lista en tupla

Importante

- **AL SER INMUTABLES** Las tuplas **no permiten** métodos como `.append()`, `.remove()` o `.sort()`.
- Son útiles para representar datos **constantes**, como coordenadas, fechas, claves de diccionarios, etc.

```
In [ ]: # Crear una tupla
coordenadas = (10.5, 20.3)
print(coordenadas) # Muestra la tupla completa
```

```
In [ ]: # Acceder a elementos por índice
print(coordenadas[0]) # Primer elemento: 10.5
print(coordenadas[1]) # Segundo elemento: 20.3
```

```
In [ ]: # Longitud de la tupla
print(len(coordenadas)) # 2
```

```
In [ ]: # Desempaquetar tupla en variables
x, y = coordenadas
print(x) # 10.5
print(y) # 20.3
```

```
In [ ]: # Concatenar tuplas
nueva = coordenadas + (30.7,)
print(nueva) # (10.5, 20.3, 30.7)
```

```
In [ ]: # Repetir tupla
repetida = coordenadas * 2
print(repetida) # (10.5, 20.3, 10.5, 20.3)
```

```
In [ ]:
```

```
In [ ]: # Ya en modo EPIC
mi_lista = (1, 1, 2, 3, 5, 8)
```

```
for factor in range(4):
    repetida = mi_lista * factor
    print(repetida)
    print("_" * 35)
```

Intenta cambiar la línea

```
repetida = mi_lista * factor
```

Por:

```
mi_lista = mi_lista * factor
```

Observa qué ocurre y razona por qué.

Pista:

- No estás modificando la tupla original, solo reasignando el nombre.

- Las tuplas son INMUTABLES: no puedes cambiar sus elementos una vez creadas.
- Pero sí puedes hacer que 'mi_lista' apunte a otra tupla.

```
In [ ]: # Verificar si un valor está en la tupla, USAMOS EL OPERADOR in
IS_IN = 20.3 in coordenadas #Fijate, le falta solo el ? para ser lenguaje hu
print(f'Está 20.3 en la tupla --> {IS_IN}')
```

```
In [ ]: IS_IN = 99 in coordenadas
print(f'Está 99 en la tupla --> {int(IS_IN)}') # False (convertido a entero)
```

```
In [ ]: # conversión explícita de booleanos,
# True convierte al entero 1.
# False convierte al entero 0

print(int(True)) # 1
print(int(False)) # 0
```

```
In [ ]: # Convertir una lista en tupla
lista = [1, 2, 3]
tupla_desde_lista = tuple(lista)
print(f'Tupla desde lista {tupla_desde_lista} tipo:{type(tupla_desde_lista)}')
```

```
In [ ]: # Se puede crear una tupla sin paréntesis??? Pues si
tupla = 10, 11, 12, 13
print(f'Tupla {tupla} tipo:{type(tupla)}')
```

```
In [ ]: # Convertir tupla en lista
lista_desde_tupla = list(tupla)
print(f'Lista desde tupla {lista_desde_tupla} tipo:{type(lista_desde_tupla)}')
```

```
In [ ]: # Tupla de varias coordenadas
coordenadas_multiples = ((1.0, 2.0), (3.5, 4.2), (5.1, 6.3))
print(f'Tupla de coordenadas múltiples: {coordenadas_multiples}')
print("_" * 35)
```

```
print(f'Primera coordenada: {coordenadas_multiples[0]}')
print("_" * 35)

print(f'Segunda coordenada: {coordenadas_multiples[1]}')
print("_" * 35)

print(f'Tercera coordenada: {coordenadas_multiples[2]}')
print("_" * 35)
```

Ejercicios guiados: Tuplas en Python

Nivel 1: Creación y acceso

Ejercicio 1.1

Crea una tupla llamada `coordenadas` con dos números decimales. Imprime la tupla completa.

Ejercicio 1.2

Accede al primer y segundo elemento de la tupla `coordenadas` e imprímelos por separado.

Ejercicio 1.3

Imprime la longitud de la tupla `coordenadas`.

In []: *#tu codigo aqui*

Nivel 2: Desempaquetado y operaciones básicas

Ejercicio 2.1

Desempaqueta la tupla `coordenadas` en dos variables llamadas `x` e `y`. Imprime cada una.

Ejercicio 2.2

Crea una nueva tupla que sea la concatenación de `coordenadas` con otra tupla que contenga un solo número.

Ejercicio 2.3

Crea una tupla llamada `repetida` que repita dos veces los elementos de `coordenadas`.

In []: *#tu codigo aqui*

Nivel 3: Verificación y conversión

Ejercicio 3.1

Verifica si el número `20.3` está en la tupla `coordenadas` y guarda el resultado en una variable.

Ejercicio 3.2

Convierte el resultado anterior (booleano) en un entero e imprímelo.

Ejercicio 3.3

Convierte una lista `[1, 2, 3]` en una tupla y muestra su tipo.

```
In [ ]: #tu codigo aqui
```

Nivel 4: Creación sin paréntesis y conversión inversa

Ejercicio 4.1

Crea una tupla llamada `tupla` con cuatro números enteros, sin usar paréntesis.

Ejercicio 4.2

Convierte la tupla `tupla` en una lista y muestra su tipo.

```
In [ ]: #tu codigo aqui
```

Diccionarios en Python

Los diccionarios son estructuras de datos que permiten almacenar pares **clave-valor**. Son **mutables**, lo que significa que se pueden modificar después de su creación.

Características principales

- Se definen con llaves `{}`.
- Cada elemento tiene una **clave** (única) y un **valor** asociado.
- Se accede a los valores mediante su clave, no por posición.
- Son útiles para representar datos estructurados, como registros, configuraciones o entidades con atributos.

Operaciones comunes

Operación	Código	Descripción
Crear diccionario	<pre>persona = {"nombre": "Ana", "edad": 30}</pre>	Diccionario con dos pares clave-valor

Operación	Código	Descripción
Acceder a un valor	<code>persona["nombre"]</code>	Devuelve <code>"Ana"</code>
Modificar un valor	<code>persona["edad"] = 31</code>	Cambia el valor asociado a la clave <code>"edad"</code>
Agregar un nuevo par	<code>persona["ciudad"] = "Sevilla"</code>	Añade una nueva clave con su valor
Eliminar un par	<code>del persona["edad"]</code>	Elimina la clave <code>"edad"</code> y su valor
Obtener claves	<code>persona.keys()</code>	Devuelve una vista de todas las claves
Obtener valores	<code>persona.values()</code>	Devuelve una vista de todos los valores
Obtener pares clave-valor	<code>persona.items()</code>	Devuelve una vista de tuplas (clave, valor)
Verificar existencia	<code>"nombre" in persona</code>	Devuelve <code>True</code> si la clave existe
Longitud	<code>len(persona)</code>	Número de pares clave-valor

Iterar sobre diccionarios (ES PRONTO PORQUE AUN NO HEMOS VISTO LOS BUCLES!!!), pero toca aquí porque es una estructura de datos

Para recorrer un diccionario, podemos usar un bucle `for`. Existen varias formas:

Iterar solo sobre las claves

```
for clave in persona:
    print(clave, persona[clave])
```

Iterar sobre claves y valores usando items() (recomendado)

```
for clave, valor in persona.items():
    print(f"{clave}: {valor}")
```

Por qué necesitamos `clave` y `valor`

- `clave` nos permite identificar cada elemento dentro del diccionario.
- `valor` es el contenido asociado a esa clave, que queremos procesar o mostrar.
- `items()` devuelve pares `(clave, valor)`, evitando tener que acceder al valor manualmente con `persona[clave]`.

```
In [ ]: # Diccionario vacío
jugadores = {}
```

```
In [ ]: # Añadir jugadores
jugadores["lamine"] = {
    "equipo": "FC Barcelona",
    "posición": "Delantero",
    "nombre": "Lamine Yamal",
    "número": 10
}
```

```
In [ ]: jugadores["nico"] = {
    "equipo": "Athletic Club",
    "posición": "Extremo",
    "nombre": "Nico Williams",
    "número": 10
}
jugadores["isco"] = {
    "equipo": "Real Betis",
    "posición": "Mediocentro",
    "nombre": "Don Francisco Alarcón",
    "número": 22
}

jugadores["mbappe"] = {
    "equipo": "Real Madrid",
    "posición": "Delantero",
    "nombre": "Kylian Mbappé",
    "número": 10
}

jugadores["griezmann"] = {
    "equipo": "Atlético de Madrid",
    "posición": "Delantero",
    "nombre": "Antoine Griezmann",
    "número": 7
}

jugadores["courtois"] = {
    "equipo": "Real Madrid",
    "posición": "Portero",
    "nombre": "Thibaut Courtois",
    "número": 1
}

jugadores["bartra"] = {
    "equipo": "Real Betis",
    "posición": "Defensa central",
    "nombre": "Marc Bartra",
    "número": 5
}
```

```
In [ ]: # Obtener las claves del diccionario
print("Claves del diccionario:", jugadores.keys())
```

```
print("----" *50) #Linea vacia
```

```
In [ ]: # Imprimimos (accedemos) a cada clave del diccionario y a los valores almacenados
for clave, info in jugadores.items():
    print(clave)
    print(info)
    print('-----')
```

```
In [ ]: # Imprimir jugador a jugador pero dando formato
for clave, info in jugadores.items():
    print(f"Jugador ID: {clave}")
    print(f"Nombre: {info['nombre']}") #Son los "nombres de los campos que tenemos en el diccionario
    print(f"Equipo: {info['equipo']}")
    print(f"Posición: {info['posición']}")
    print(f"Número: {info['número']}")
    print("-----")
```

```
In [ ]: #Y si no conocieramos los nombres de los campos???

for clave, info in jugadores.items():
    print(f"Jugador ID: {clave}")
    for campo, valor in info.items(): # Iteramos sobre cada clave-valor del diccionario
        print(f"{campo.upper()}: {valor}")
    print("-----")
```

```
In [ ]: # Encontrar cuántos Jugadores tienen numero 10

# Obtener todos los números de los jugadores en una lista
numeros = [info["número"] for info in jugadores.values()]

"""
El bucle for va a sacar value para cada jugador y lo va a llamar info
Para cada info sacado, extrae lo que haya en info["numero"]
Y como todo va envuelto en un [ ] genera una lista que va creciendo según va iterando
"""

#Y como numeros es una lista, puedo contar las veces que sale un valor determinado
print(f'Hay {numeros.count(10)} jugadores con el 10')
```

```
In [ ]: # Informar sobre los jugadores con el 22
print("Jugadores con número 22:")
for clave, info in jugadores.items():
    if info["número"] == 22:
        print("-", info["nombre"])
```

```
In [ ]: # Buscar si algún jugador juega en el Betis
print("\nJugadores en el Real Betis:") #Observa el \n para meter una línea nueva

for clave, info in jugadores.items():
    if info["equipo"] == "Real Betis":
        print("-", info["nombre"])
```

```
In [ ]: # Cambiar el equipo de Mbappé al Betis
jugadores["mbappe"]["equipo"] = "Real Betis"

# Verificamos el cambio
print(f"Mbappé ahora juega en: {jugadores['mbappe']['equipo']}")
```

Conjuntos en Python

Los conjuntos son estructuras de datos que permiten almacenar elementos **únicos** y **no ordenados**. Son **mutables**, lo que significa que se pueden modificar después de su creación.

Características principales

- Se definen con llaves `{}` o usando `set()`.
- No permiten elementos duplicados.
- No garantizan orden ni acceso por índice.
- Son útiles para eliminar duplicados, verificar pertenencia y realizar operaciones matemáticas como unión e intersección.

Operaciones comunes

Operación	Código	Descripción
Crear conjunto	<code>numeros = {1, 2, 3}</code>	Conjunto con tres elementos únicos
Añadir elemento	<code>numeros.add(4)</code>	Añade el número 4 al conjunto
Eliminar elemento	<code>numeros.remove(2)</code>	Elimina el número 2 del conjunto
Eliminación segura	<code>numeros.discard(99)</code>	Elimina sin error si el elemento no existe
Unión	<code>a b</code>	Elementos presentes en <code>a</code> o <code>b</code>
Intersección	<code>a & b</code>	Elementos comunes entre <code>a</code> y <code>b</code>
Diferencia	<code>a - b</code>	Elementos en <code>a</code> que no están en <code>b</code>
Verificar existencia	<code>3 in numeros</code>	Devuelve <code>True</code> si el número 3 está en el conjunto
Longitud	<code>len(numeros)</code>	Número de elementos únicos en el conjunto

Importante

- Los conjuntos **no permiten indexación** como listas o tuplas.

- Para recorrerlos, se usa un bucle `for`.
- Si intentas añadir un elemento duplicado, no se genera error, pero no se añade.

```
In [ ]: ## Ejemplos básicos con conjuntos
```

```
# Crear un conjunto  
numeros = {1, 2, 3, 4}  
print(numeros)
```

```
In [ ]: # Añadir un elemento  
numeros.add(5)  
print(numeros)
```

```
In [ ]: # Eliminar un elemento  
numeros.remove(2)  
  
print(numeros)
```

```
In [ ]: # Verificar existencia  
print(3 in numeros) # True
```

```
In [ ]: print(10 in numeros) # False
```

```
In [ ]: # Unión de conjuntos  
pares = {2, 4, 6}  
impares = {1, 3, 5}  
todos = pares | impares  
print(f'Unión: {todos}')
```

```
In [ ]: # Intersección  
print(f'Conjunto 1: {numeros}')
```

```
print(f'Conjunto 2: {pares}')
```

```
print(f'Intersección: {numeros & pares}')
```

```
In [ ]: # Diferencia  
print(f'Conjunto 1: {numeros}')
```

```
print(f'Conjunto 2: {pares}')
```

```
print(f'Diferencia: {numeros - pares}')
```

```
In [ ]: # Unir diccionarios usando conjuntos  
# Los conjuntos pueden usarse para combinar claves o  
# valores de varios diccionarios, eliminando duplicados.
```

```
### Ejemplo 1: Unir claves de dos diccionarios
```

```
dic1 = {"a": 1, "b": 2, "c": 3}  
dic2 = {"b": 4, "d": 5}
```

```
# Unir claves  
claves_unidas = set(dic1.keys()) | set(dic2.keys())  
print(f'Claves únicas: {claves_unidas}')
```

```
In [ ]: valores_unicos = set(dic1.values()) | set(dic2.values()) # OR --> | une
print(f'Valores únicos: {valores_unicos}')
```

```
In [ ]: claves_comunes = set(dic1.keys()) & set(dic2.keys()) # AND --> & compara
print(f'Claves comunes: {claves_comunes}')
```

Ejercicio guiado: Diccionarios con aves

Nivel 1: Creación de diccionarios con estructura

Objetivo: Crear dos diccionarios que representen aves con distintos niveles de detalle.

1. Crea un diccionario llamado `aves1` que contenga 5 aves.

Cada ave debe tener como claves:

- `nombre_comun`
- `nombre_cientifico`
- `genero`

2. Crea un segundo diccionario llamado `aves2` con otras 3 aves.

Este debe tener los mismos campos que `aves1`, **más** un campo adicional:

- `en_peligro_extincion` (tipo `bool`)

Referencia de aves

Esta es una selección de aves comunes y algunas en peligro de extinción:

ID	Nombre común	Nombre científico	Género	En peligro
ave1	Abejaruco europeo	<i>Merops apiaster</i>	Merops	No
ave2	Estornino negro	<i>Sturnus unicolor</i>	Sturnus	No
ave3	Pinzón azul	<i>Fringilla polatzeki</i>	Fringilla	Sí
ave4	Águila imperial	<i>Aquila adalberti</i>	Aquila	Sí
ave5	Abubilla común	<i>Upupa epops</i>	Upupa	No
ave6	Milano negro	<i>Milvus migrans</i>	Milvus	No
ave7	Águila real	<i>Aquila chrysaetos</i>	Aquila	No
ave8	Verderón común	<i>Chloris chloris</i>	Chloris	No

```
In [ ]: # Diccionario aves1 con 5 aves
aves1 = {
    "ave1": {
        "nombre_comun": "Abejaruco europeo",
        "nombre_cientifico": "Merops apiaster",
        "genero": "Merops"
```

```

    },
    "ave2": {
        "nombre_comun": "Estornino negro",
        "nombre_cientifico": "Sturnus unicolor",
        "genero": "Sturnus"
    },
    "ave3": {
        "nombre_comun": "Abubilla común",
        "nombre_cientifico": "Upupa epops",
        "genero": "Upupa"
    },
    "ave4": {
        "nombre_comun": "Águila real",
        "nombre_cientifico": "Aquila chrysaetos",
        "genero": "Aquila"
    },
    "ave5": {
        "nombre_comun": "Milano negro",
        "nombre_cientifico": "Milvus migrans",
        "genero": "Milvus"
    }
}

# Diccionario aves2 con 3 aves y campo adicional
aves2 = {
    "ave6": {
        "nombre_comun": "Pinzón azul",
        "nombre_cientifico": "Fringilla polatzeki",
        "genero": "Fringilla",
        "en_peligro_extincion": True
    },
    "ave7": {
        "nombre_comun": "Águila imperial",
        "nombre_cientifico": "Aquila adalberti",
        "genero": "Aquila",
        "en_peligro_extincion": True
    },
    "ave8": {
        "nombre_comun": "Verderón común",
        "nombre_cientifico": "Chloris chloris",
        "genero": "Chloris",
        "en_peligro_extincion": False
    }
}

for clave, info in aves1.items():
    print(f"Ave ID: {clave}")
    for campo, valor in info.items(): # Iteramos sobre cada clave-valor del
        print(f"{campo.upper()}: {valor}")
    print("-----")
for clave, info in aves2.items():
    print(f"Ave ID: {clave}")
    for campo, valor in info.items(): # Iteramos sobre cada clave-valor del
        print(f"{campo.upper()}: {valor}")
    print("-----")

```

Genera una nuevo diccionario con las aves del mismo genero en aves1 y aves2

```
In [ ]: # Crear conjunto de géneros comunes

generos_en_1 = set()
for info in aves1.values():
    generos_en_1.add(info["genero"])

generos_en_2 = set()
for info in aves2.values():
    generos_en_2.add(info["genero"])

generos_comunes = generos_en_1 & generos_en_2

#generos_comunes = {info["genero"] for info in aves1.values()} & {info["genero"] for info in aves2.values()}

# Observa como estamos haciendo una operacion & entre dos expresiones dentro de llaves es la operacion interseccion entre conjuntos.
# El resultado generos_comunes es de tipo set (conjunto)

print(type(generos_comunes))
```

```
In [ ]: # Crear aves3 con aves de aves1 y aves2 que tengan género en común
aves3 = {}

# Añadir aves de aves1
for clave, info in aves1.items():
    if info["genero"] in generos_comunes:
        aves3[clave] = info

# Añadir aves de aves2
for clave, info in aves2.items():
    if info["genero"] in generos_comunes:
        aves3[clave] = info

#Imprimimos aves3 a ver si está bien!
for clave, info in aves3.items():
    print(f"Ave ID: {clave}")
    for campo, valor in info.items(): # Iteramos sobre cada clave-valor del diccionario
        print(f"{campo.upper():} {valor}")
    print("-----")
```

Como vemos, de ave4 no tenemos datos de si está o no en peligro de extincion así que ese campo no está completo, pensemos en descartar esa informacion por ejemplo.

```
In [ ]: # Paso 1: Identificar los campos comunes entre todas las aves en aves3

# Forma compacta en una sola linea
# campos_comunes = set.intersection(*(set(info.keys()) for info in aves3.values()))
```

```

# Versión didáctica con for: construimos un set de claves por cada ave
conjuntos_campos = []
for info in aves3.values():
    conjuntos_campos.append(set(info.keys()))

# Si no hay aves, no hay campos comunes
if len(conjuntos_campos) == 0:
    campos_comunes = set()
else:
    # Empezamos con los campos de la primera ave
    campos_comunes = conjuntos_campos[0].copy()

    # Nos quedamos solo con lo que aparece en todas las demás
    for campos_ave in conjuntos_campos[1:]:
        campos_comunes = campos_comunes.intersection(campos_ave)

print(f"Campos comunes: {campos_comunes}")

# Paso 2: Crear una nueva versión filtrada de aves3 con solo los campos comunes
aves3_filtradas = {}

for clave, info in aves3.items():
    aves3_filtradas[clave] = {campo: info[campo] for campo in campos_comunes}

# Paso 3: Mostrar aves3_filtradas
print("Aves3 filtradas con solo campos comunes:")
for clave, info in aves3_filtradas.items():
    print(f"Ave ID: {clave}")
    for campo, valor in info.items():
        print(f"{campo.upper():} {valor}")
    print("-----")

```

CUESTIONARIO SESION 2

Ejercicios practicos: Estructuras de datos en Python

En estos ejercicios pondras en practica listas, tuplas, diccionarios y conjuntos.

Ejercicios de autoevaluacion

1. Crea una lista `numeros = [3, 7, 2, 9, 5]`.

- Imprime el primer y ultimo elemento.
- Imprime una sublista con los 3 elementos centrales.
- Imprime la lista invertida usando slicing.

In []: *#tu codigo aqui*

2. Trabaja con una lista de compras.

- Define `compras = ["pan", "leche", "huevos"]`.
- Agrega `"fruta"`.
- Elimina `"leche"`.
- Imprime la lista final y su longitud.

In []: *#tu codigo aqui*

3. Usa una tupla `datos = ("Laura", 21, "Fisica")`.

- Desempaqueta en tres variables.
- Imprime cada variable con un texto descriptivo.
- Muestra el tipo de `datos`.

In []: *#tu codigo aqui*

4. Trabaja con diccionarios.

- Define `alumno = {"nombre": "Ana", "edad": 20, "curso": "Python"}`.
- Cambia la edad a 21.
- Agrega la clave `"ciudad"` con valor `"Madrid"`.
- Imprime solo las claves y luego solo los valores.

In []: *#tu codigo aqui*

5. Recorre una lista y genera otra.

- Define `notas = [5, 8, 4, 9, 6]`.
- Crea una lista `aprobadas` con las notas mayores o iguales a 6.
- Imprime `aprobadas` y cuantas notas aprobadas hay.

In []: *#tu codigo aqui*

6. Trabaja con conjuntos (`set`).

- Define `A = {1, 2, 3, 4}` y `B = {3, 4, 5, 6}`.
- Imprime union, interseccion y diferencia `A - B`.
- Comprueba si `2` pertenece a `B`.

In []: *#tu codigo aqui*

7. Convierte estructuras.

- Define una lista `lista = ["a", "b", "a", "c", "b"]`.
- Convierte a conjunto para eliminar repetidos.
- Convierte de nuevo a lista.
- Imprime las tres estructuras para comparar.

```
In [ ]: #tu codigo aqui
```

8. Crea un diccionario a partir de dos listas.

- Define `claves = ["id", "nombre", "nota"]` y `valores = [1, "Luis", 8.5]`.
- Crea el diccionario con `zip`.
- Imprime el diccionario y accede al valor de `"nota"`.

```
In [ ]: #tu codigo aqui
```

9. Filtra elementos unicos de una lista manteniendo orden.

- Define `datos = [4, 2, 4, 3, 2, 1, 3]`.
- Crea una nueva lista `unicos` recorriendo `datos` y agregando solo si no estaba antes.
- Imprime `unicos`.

```
In [ ]: #tu codigo aqui
```

10. Mini reto integrador (sin usar `def`).

- Define una lista de diccionarios llamada `alumnos` con 3 alumnos y sus notas.
- Recorre la estructura y crea una lista `nombres_aprobados` con nota ≥ 6 .
- Imprime `nombres_aprobados` y el total de aprobados.

```
In [ ]: #tu codigo aqui
```

```
In [ ]: # SOLUCIONARIO INTEGRADO – ESTRUCTURAS DE DATOS
# Incluye enunciado + solucion de referencia para cada ejercicio.
```

```
print("=" * 70)
print("EJERCICIO 1")
print("Enunciado: operaciones basicas con lista y slicing.")
numeros = [3, 7, 2, 9, 5]
print("primero:", numeros[0])
print("ultimo:", numeros[-1])
print("centrales:", numeros[1:4])
print("invertida:", numeros[::-1])
```

```

print("\n" + "=" * 70)
print("EJERCICIO 2")
print("Enunciado: agregar y eliminar elementos en lista.")
compras = ["pan", "leche", "huevos"]
compras.append("fruta")
compras.remove("leche")
print("compras final:", compras)
print("longitud:", len(compras))

print("\n" + "=" * 70)
print("EJERCICIO 3")
print("Enunciado: desempaquetar una tupla.")
datos = ("Laura", 21, "Fisica")
nombre, edad, carrera = datos
print("Nombre:", nombre)
print("Edad:", edad)
print("Carrera:", carrera)
print("Tipo de datos:", type(datos))

print("\n" + "=" * 70)
print("EJERCICIO 4")
print("Enunciado: modificar y consultar diccionario.")
alumno = {"nombre": "Ana", "edad": 20, "curso": "Python"}
alumno["edad"] = 21
alumno["ciudad"] = "Madrid"
print("claves:", list(alumno.keys()))
print("valores:", list(alumno.values()))

print("\n" + "=" * 70)
print("EJERCICIO 5")
print("Enunciado: filtrar notas aprobadas en una lista.")
notas = [5, 8, 4, 9, 6]
aprobadas = []
for n in notas:
    if n >= 6:
        aprobadas.append(n)
print("aprobadas:", aprobadas)
print("cantidad aprobadas:", len(aprobadas))

print("\n" + "=" * 70)
print("EJERCICIO 6")
print("Enunciado: operaciones con conjuntos.")
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("union:", A | B)
print("interseccion:", A & B)
print("diferencia A-B:", A - B)
print("2 pertenece a B?:", 2 in B)

print("\n" + "=" * 70)
print("EJERCICIO 7")
print("Enunciado: convertir lista->set->lista.")
lista = ["a", "b", "a", "c", "b"]
conjunto = set(lista)
lista_sin_repetidos = list(conjunto)

```

```

print("lista original:", lista)
print("set:", conjunto)
print("lista final:", lista_sin_repetidos)

print("\n" + "=" * 70)
print("EJERCICIO 8")
print("Enunciado: crear diccionario con zip.")
claves = ["id", "nombre", "nota"]
valores = [1, "Luis", 8.5]
registro = dict(zip(claves, valores))
print("diccionario:", registro)
print("nota:", registro["nota"])

print("\n" + "=" * 70)
print("EJERCICIO 9")
print("Enunciado: unicos manteniendo orden.")
datos = [4, 2, 4, 3, 2, 1, 3]
unicos = []
for x in datos:
    if x not in unicos:
        unicos.append(x)
print("unicos:", unicos)

print("\n" + "=" * 70)
print("EJERCICIO 10")
print("Enunciado: mini reto con lista de diccionarios.")
alumnos = [
    {"nombre": "Ana", "nota": 7},
    {"nombre": "Luis", "nota": 5},
    {"nombre": "Marta", "nota": 9}
]
nombres_aprobados = []
for a in alumnos:
    if a["nota"] >= 6:
        nombres_aprobados.append(a["nombre"])
print("nombres_aprobados:", nombres_aprobados)
print("total aprobados:", len(nombres_aprobados))

```